TP 1 : Résolution du Sac à Dos 0-1

Des Heuristiques Gloutonnes au Branch and Bound – Analyse de Scalabilité

Niveau : 4e Année Ingénieur en Intelligence Artificielle
Module : Recherche Opérationnelle

Année Universitaire 2026–2027

---

1 Introduction et Données du Problème

Le problème du sac à dos binaire consiste à maximiser la valeur cumulée d'objets embarqués sous contrainte d'une capacité maximale W :

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{s.c.} \quad \sum_{i=1}^{n} w_i x_i \leq W, \quad x_i \in \{0,1\} \tag{1}$$

L'instance de référence étudiée (issue de l'arbre de décision) possède une capacité $W = 10$ et 4 objets triés par ratio décroissant ($r_i = v_i/w_i$) :

Objet (i)	Valeur (v_i)	Poids (w_i)	Ratio ($r_i = v_i/w_i$)
1	5	2	2.50
4	6	3	2.00
3	7	4	1.75
2	8	5	1.60

L'arbre de référence montre qu'une heuristique initiale fournit une borne inférieure $B_{\text{inf}} = 18$. L'optimum théorique attendu est $Z_{\max} = 19$ avec le sous-ensemble d'objets $\{1, 4, 2\}$.

2 Partie I : Heuristiques Constructives Gloutonnes (Fournie)

Les trois heuristiques gloutonnes ci-dessous sont déjà écrites. Elles construisent une solution admissible en triant les éléments selon un critère local prédéfini :

```python
def heuristic_max_value(items, capacity):
    """Heuristique 1 : Tri par profit brut decroissant (v_i)"""
    sorted_items = sorted(items, key=lambda x: x["v"], reverse=True)
    total_val, rem_w, selected = 0, capacity, []
    for item in sorted_items:
        if item["w"] <= rem_w:
            selected.append(item["id"])
            total_val += item["v"]
            rem_w -= item["w"]
    return total_val, capacity - rem_w, selected

def heuristic_min_weight(items, capacity):
    """Heuristique 2 : Tri par poids croissant (w_i)"""
    sorted_items = sorted(items, key=lambda x: x["w"])
    total_val, rem_w, selected = 0, capacity, []
    for item in sorted_items:
        if item["w"] <= rem_w:
```

```
            selected.append(item["id"])
            total_val += item["v"]
            rem_w -= item["w"]
    return total_val, capacity - rem_w, selected

def heuristic_ratio(items, capacity):
    """Heuristique 3 : Tri par densite decroissante (v_i / w_i)"""
    sorted_items = sorted(items, key=lambda x: x["v"] / x["w"], reverse=True)
    total_val, rem_w, selected = 0, capacity, []
    for item in sorted_items:
        if item["w"] <= rem_w:
            selected.append(item["id"])
            total_val += item["v"]
            rem_w -= item["w"]
    return total_val, capacity - rem_w, selected
```

Questions d'Analyse (Partie I)

1. Exécutez les trois heuristiques sur l'instance $W = 10$. Quelle heuristique permet d'obtenir la borne $B_{\text{inf}} = 18$?
2. Justifiez pourquoi l'heuristique basée sur le ratio fournit en pratique de meilleures bornes inférieures pour initialiser une recherche arborescente.

3 Partie II : Borne Supérieure par Relaxation Continue (À Compléter)

Pour élaguer l'arbre de recherche, on calcule la borne supérieure B_{sup} par la résolution du sac à dos fractionnaire ($x_i \in [0, 1]$) sur les objets restants. Dès qu'un objet dépasse la capacité restante K, on en prend une fraction continue pour saturer le sac.

Code à Compléter : Calcul de la Borne Continue

```
def compute_upper_bound(idx, current_val, remaining_cap, sorted_items):
    """
    Parametres :
    - idx : indice de l'objet courant a partir duquel evaluer (Rest)
    - current_val : profit cumule actuel (Benf)
    - remaining_cap : capacite restante disponible (K)
    - sorted_items : liste des objets tries par ratio decroissant
    """
    if remaining_cap < 0:
        return -1.0  # Noeud inadmissible

    bsup = float(current_val)
    cap = remaining_cap

    # ------------------ A COMPLETER PAR L'ETUDIANT ------------------
    # TODO 1 :
    # Parcourir les objets a partir de 'idx' jusqu'a la fin :
    #    - Si le poids de l'objet <= cap :
    #        Ajouter sa valeur a bsup et decrementer cap
    #    - Sinon :
    #        Ajouter la fraction (cap * (v / w)) a bsup et quitter la boucle
    pass
    # ----------------------------------------------------------------

    return bsup
```

Questions d'Analyse (Partie II)

1. Calculez manuellement la borne B_{sup} à la racine et démontrez qu'elle est égale à 19.6. Précisez l'objet critique et sa fraction prise.
2. Calculez manuellement B_{sup} au nœud droit $x_1 = 0$ et vérifiez qu'elle vaut 17.8. Expliquez pourquoi ce nœud est immédiatement élagué face à $B_{\text{inf}} = 18$.

4 Partie III : Algorithme de Branch and Bound (À Compléter)

L'algorithme explore l'arbre de décision en profondeur (*DFS*). À chaque nœud, on applique la règle d'élagage par borne : si $B_{\text{sup}} \leq B_{\text{inf}}$, le sous-arbre est ignoré.

Code à Compléter : Moteur Branch and Bound

```
class BranchAndBound:
    def __init__(self, items, capacity, initial_lb=18):
        # Tri obligatoire par densite decroissante
        self.items = sorted(items, key=lambda x: x["v"]/x["w"], reverse=True)
        self.W = capacity
        self.best_val = initial_lb  # Borne inferieure courante (Binf)
        self.best_sol = []
        self.nodes = 0

    def solve(self):
        self._dfs(idx=0, current_val=0, rem_cap=self.W, taken=[])
        return self.best_val, self.best_sol, self.nodes

    def _dfs(self, idx, current_val, rem_cap, taken):
        self.nodes += 1

        # 1. Mise a jour si une meilleure solution entiere admissible est trouvee
        if current_val > self.best_val:
            self.best_val = current_val
            self.best_sol = list(taken)

        # Condition d'arret : toutes les variables ont ete decidees
        if idx >= len(self.items):
            return

        # ------------------ A COMPLETER PAR L'ETUDIANT ------------------
        # TODO 2 :
        # a. Calculer la borne superieure 'bsup' en appelant compute_upper_bound.
        # b. Evaluer la condition d'elagage :
        #    Si bsup <= self.best_val, interrompre l'exploration de cette branche.
        # c. Branche Gauche (x_i = 1) :
        #    Si le sac peut contenir l'objet (rem_cap >= item["w"]) :
        #       Ajouter item["id"] dans taken, appeler recursivement _dfs,
        #       puis retirer item["id"] de taken (Backtracking).
        # d. Branche Droite (x_i = 0) :
        #    Appeler recursivement _dfs sans inclure l'objet.
        pass
        # ----------------------------------------------------------------
```

Questions d'Analyse (Partie III)

1. Exécutez votre algorithme avec $B_{\text{inf}} = 18$. Vérifiez que vous obtenez l'optimum $Z_{\max} = 19$ avec les objets $\{1, 4, 2\}$. Notez le nombre total de nœuds explorés.
2. Relancez l'algorithme avec une borne naïve $B_{\text{inf}} = 0$. Comparez le nombre de nœuds explorés et commentez l'impact d'une initialisation heuristique de qualité.

5 Partie IV : Analyse de Passage à l'Échelle (Scalabilité)

Pour évaluer la robustesse du Branch and Bound, nous générons des instances aléatoires avec $n \in \{10, 15, 20, 25\}$ objets :

— $v_i \sim \mathcal{U}[10, 100]$, $w_i \sim \mathcal{U}[5, 50]$.

— Capacité du sac : $W = 0.5 \times \sum_{i=1}^{n} w_i$.

```python
import random, time

def generate_instance(n, seed=42):
    random.seed(seed)
    items = [{"id": i, "v": random.randint(10, 100), "w": random.randint(5, 50)} for i in range(n)]
    W = int(0.5 * sum(x["w"] for x in items))
    return items, W
```

Travail Expérimental

1. Complétez le tableau suivant en mesurant les performances pour chaque taille n :

Taille (n)	**Arbre exhaustif ($2^{n+1}-1$)**	**Nœuds BB ($B_{\text{inf}} = 0$)**	**Nœuds BB (Glouton)**	**Temps (s)**
10	2 047			
15	65 535			
20	2 097 151			
25	67 108 863			

2. Tracez en échelle semi-logarithmique la courbe du nombre de nœuds explorés en fonction de n.
3. Évaluez l'écart d'optimalité moyen (*optimality gap*) de l'heuristique par ratio :

$$\text{Gap} = \frac{Z^* - Z_{\text{glouton}}}{Z^*} \times 100\% \tag{2}$$